



《计算机科学导论》课程

软件工程

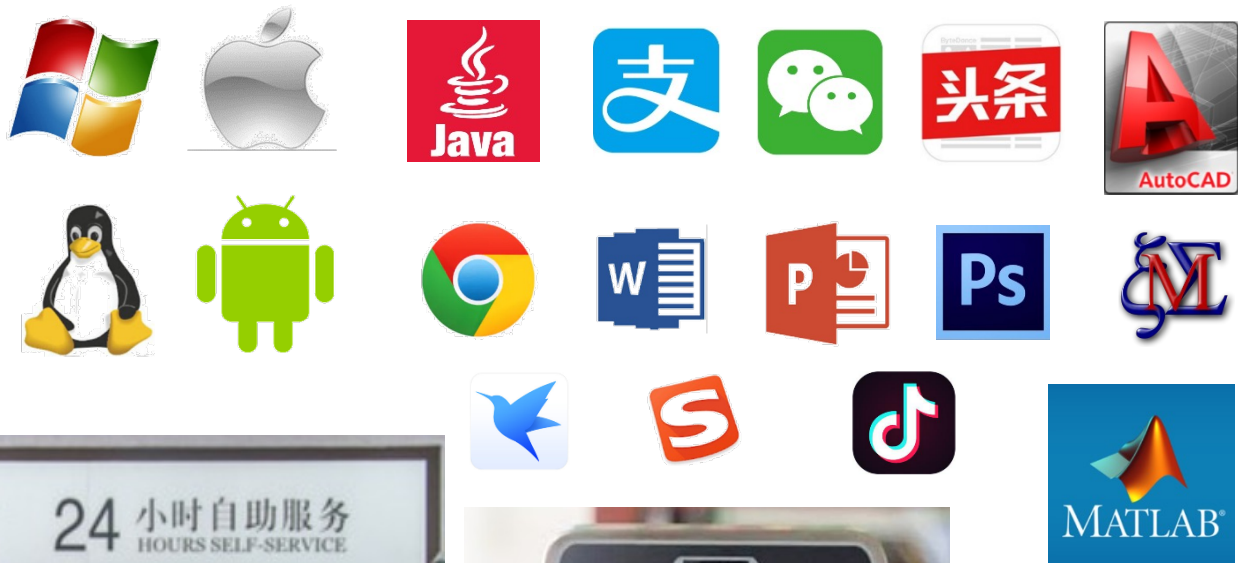
上海交通大学计算机科学导论课程组



无处不在的软件-智能门锁



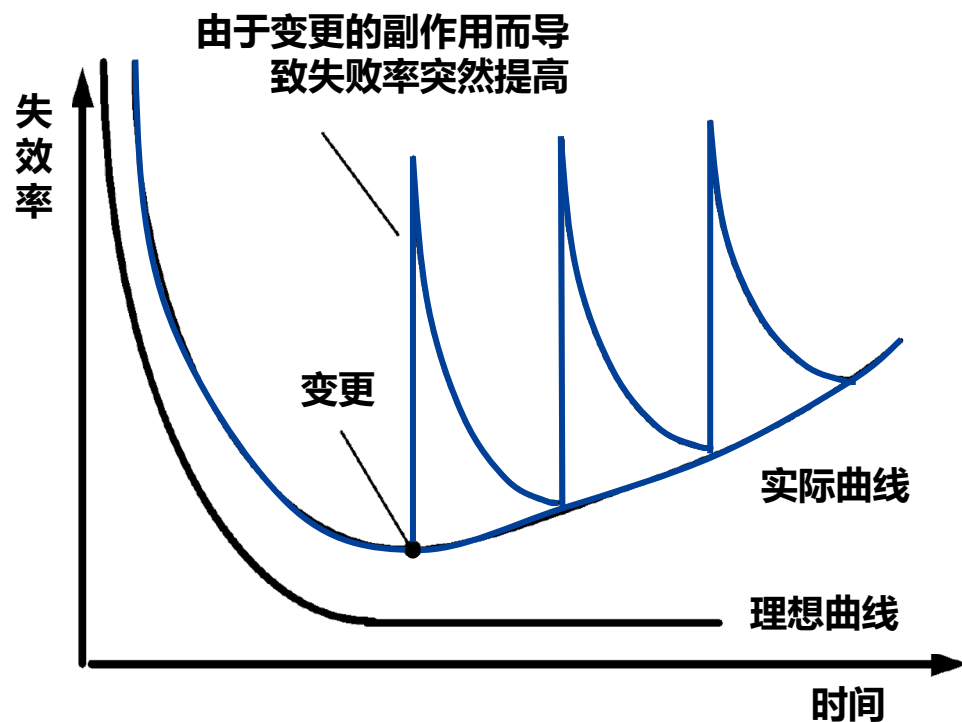
无处不在的软件



软件 (Software) 是一系列按照特定顺序组织的计算机数据和指令，是计算机中的非有形部分

软件特征

- 软件是逻辑的而不是有形的系统元件
- 软件具有与硬件完全不同的特征
 - ◆ 软件是被开发或设计的，而不是被制造的
 - ◆ 虽然软件产品正在向基于构件的组装前进，大多数软件仍然是定制的
 - ◆ 软件不会“磨损”



软件危机

软件危机是指落后的软件生产方式无法满足迅速增长的计算机软件需求，从而导致软件开发与维护过程中出现一系列严重问题的现象。

1968年，北大西洋公约组织（NATO）在联邦德国的国际学术会议创造软件危机（Software crisis）一词。



软件危机：案例

OS/360：1961-1964年，IBM公司的 OS/360，共约100万条指令，花费了5000个人年，开发总投资5亿美元，达到当时美国研究原子弹的曼哈顿计划投资20亿美元的1/4，而结果却令人沮丧，错误多达2000个以上，系统根本无法正常运行。

《人月神话》：由IBM System/360系统之父佛瑞德·布鲁克斯所著文集，全书讲解软件工程、项目管理相关课题，被誉为软件领域的圣经，内容源于作者布鲁克斯在IBM公司System/360家族和OS/360中的专案管理经验。



佛瑞德·布鲁克斯

Fred Brooks

IBM System/360系统之父
1999年图灵奖得主

软件危机：案例

◆ **美国银行信托软件系统：**美国银行1982年进入信托商业领域，并规划发展信托软件系统。计划原定预算2千万美元，开发时程9个月，预计于1984年12月31日以前完成，后来至1987年3月都未能完成该系统，期间已投入6千万美元。美国银行最终因为此系统不稳定而不得不放弃，并将340亿美元的信托帐户转移出去，并失去了6亿美元的信托生意商机。

◆ **Therac-25 放射医疗仪事故：**1986年由于软件错误导致放射过量，致2人死亡。溢出错误是导致问题的主要原因之一。



1968年秋季，NATO（北约）的科技委员会召集了近50名一流的编程人员、计算机科学家和工业界巨头，讨论和制定摆脱“软件危机”的对策。

在会议上第一次提出了软件工程（software engineering）这个概念，研究和应用如何以系统性的、规范化的、可量化的过程化方法去开发和维护软件，以及如何把经过时间考验而证明正确的管理技术和当前能够得到的最好的技术方法结合起来的学科。

柏瑞·贝姆教授（Barry W. Boehm）
美国国家工程院院士，AIAA、IEEE、ACM会士（Fellow）， “软件工程经济学之父”

B. Boehm, "Software Engineering" in IEEE Transactions on Computers, vol. 25, no. 12, pp. 1226-1241, 1976. doi: 10.1109/TC.1976.1674590

Software Engineering: The practical application of scientific knowledge in the design and construction of computer programs and the associated documentation required to develop, operate, and maintain them.

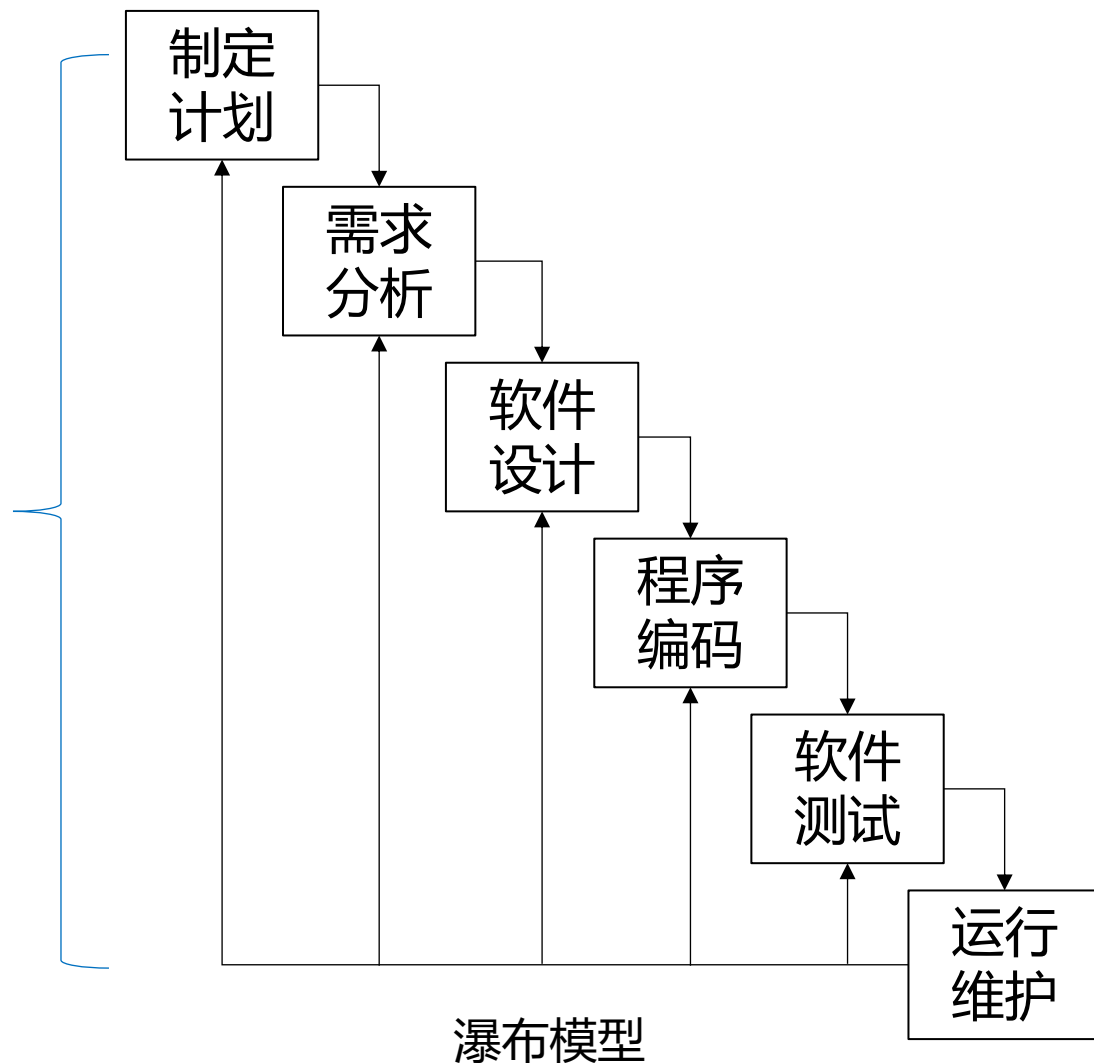
软件工程是（1）将系统化的、规范的、可度量的方法应用于软件的开发、运行和维护的过程，即将工程化应用于软件中；（2）在（1）中所述方法的研究

软件生存周期是指软件产品从考虑其概念开始到产品交付使用，直到最终退役

软件开发模型是软件开发的全部过程、活动和任务的结构框架。

- 瀑布模型：自上而下，相互衔接

软件的生存周期



目录

01

软件需求分析

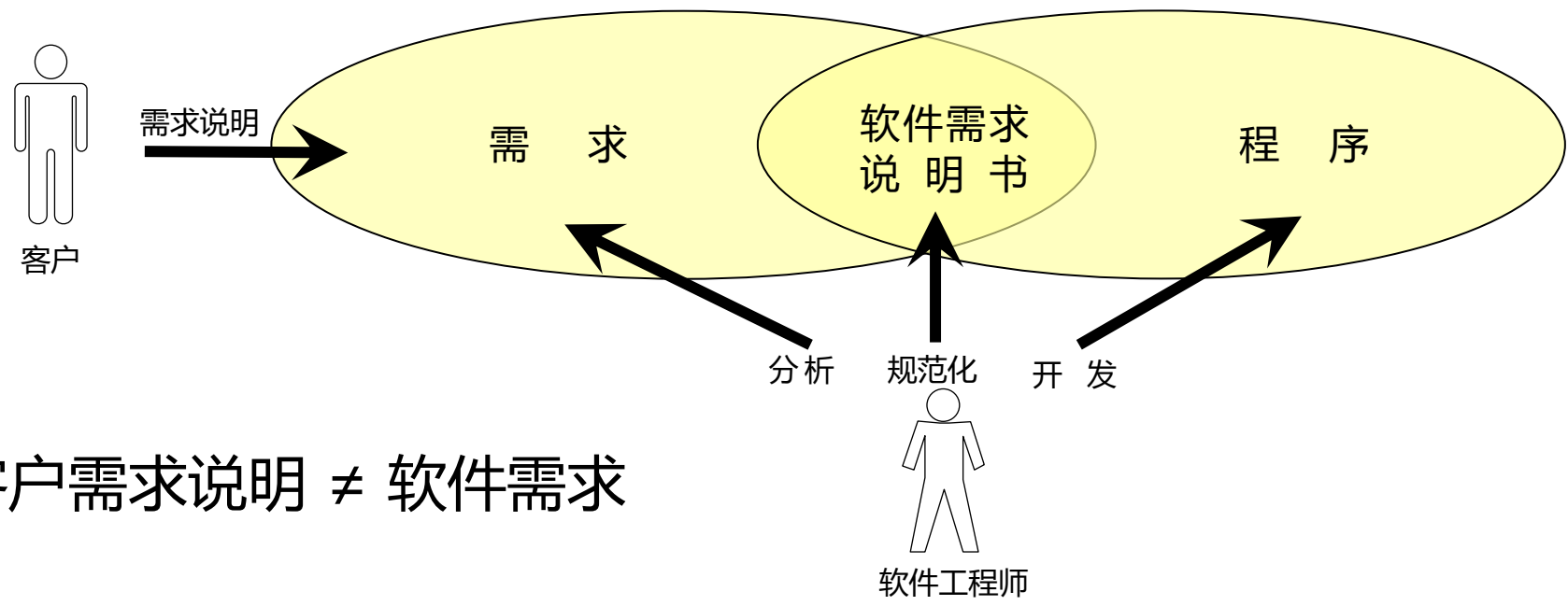
02

软件结构设计

03

软件测试策略

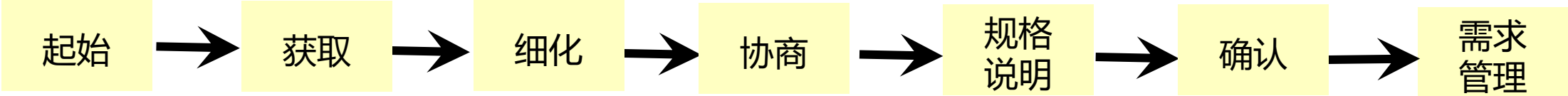
01.软件需求分析



客户需求说明 ≠ 软件需求

软件需求分析中的七个任务

向客户明确软件的目标，明确软件如何满足业务的要求 协商资源和客户要求之间的矛盾，和客户协商需求的优先级 检查规格说明保证无歧义的说明了所有的系统需求

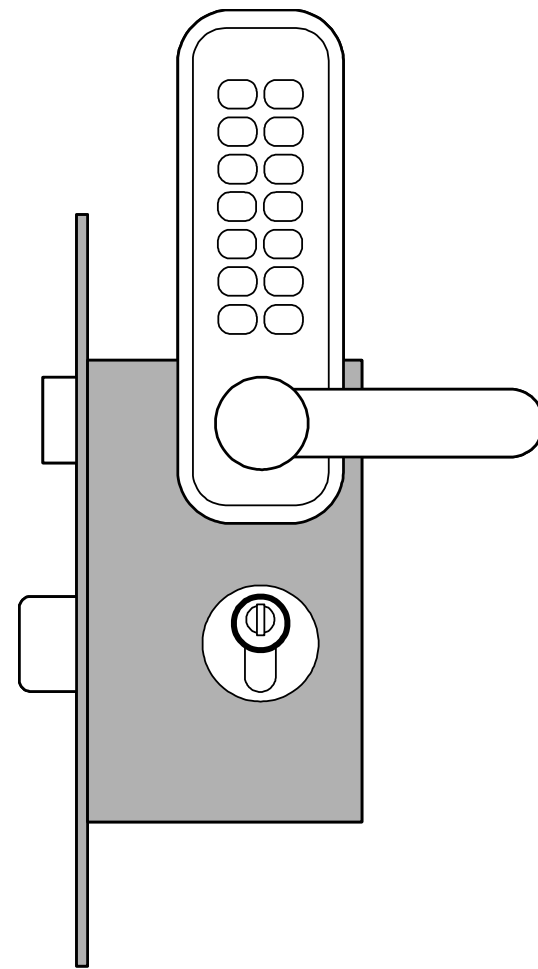
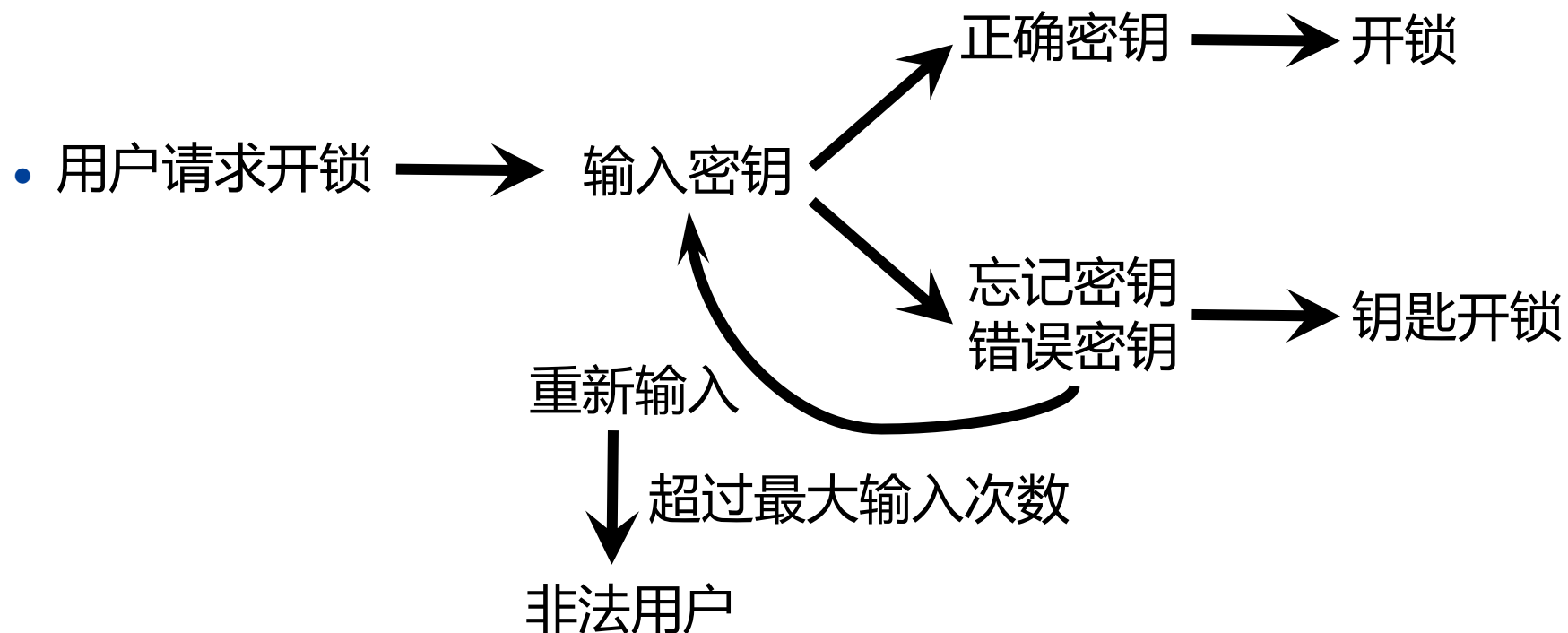


与利益相关者的初步交流，沟通存在的问题，期望解决方案等 开发精确的需求模型，由用户场景建模，解析每个用户场景提取最终用户可见的业务实体 在基于计算机系统的环境下使用术语进行规格说明 在整个生存周期中，控制和跟踪需求以及需求变更

01.软件需求分析：Safe Home Access安全门锁软件系统

用户使用场景分析（细化示例）

- 无开门请求时 → 保持门被锁状态



01.软件需求分析： Safe Home Access安全门锁软件系统

电气电子工程师学会830版本(IEEE-830)软件需求说明格式

- 以用户使用场景为基础分析
软件需求
- 概括说明软件功能需求
- 指出需求的优先级，将需求
按照优先级排序（降序）
- 需求具有唯一标识（识别号）

识别号	优先级	需求
REQ-1	5	在无开锁请求时，系统需要保持门被锁状态。当锁打开时，应启动倒计时，倒计时结束自动锁住
REQ-2	5	系统应在给出指定密钥时打开门
REQ-3	4	在输入密钥时，系统容许有限次数的错误输入，超过最大输入次数后保持门被锁状态且禁止继续输入
REQ-4	2	系统应该维护历史输入记录
REQ-5	2	系统应该允许加入新的授权用户
REQ-6	2	系统应可通过机械开关开门和锁门

目 录

01

软件需求分析

02

软件结构设计

03

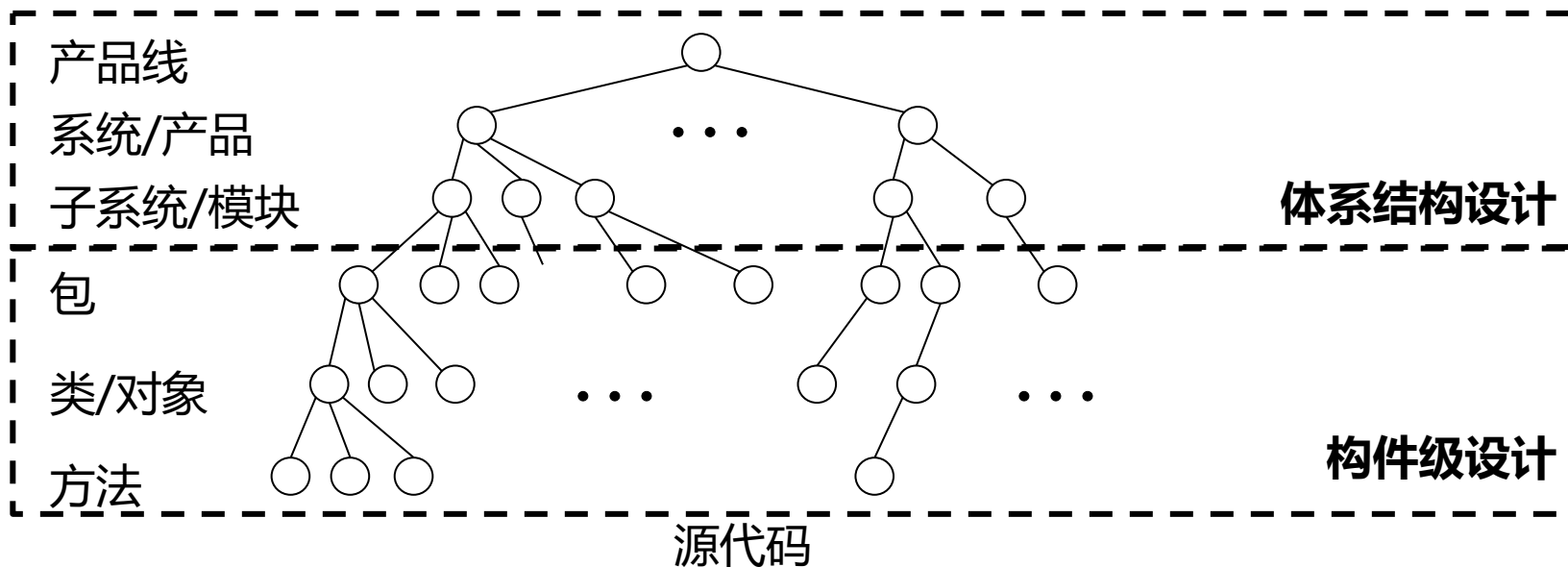
软件测试策略

02.软件结构设计

程序或计算系统的**软件系统结构**是指系统的一个或者多个结构，它包括软件构件、构件的外部可见属性以及它们之间的相互关系

体系结构设计描述软件系统如何构成以及其构件如何一起工作

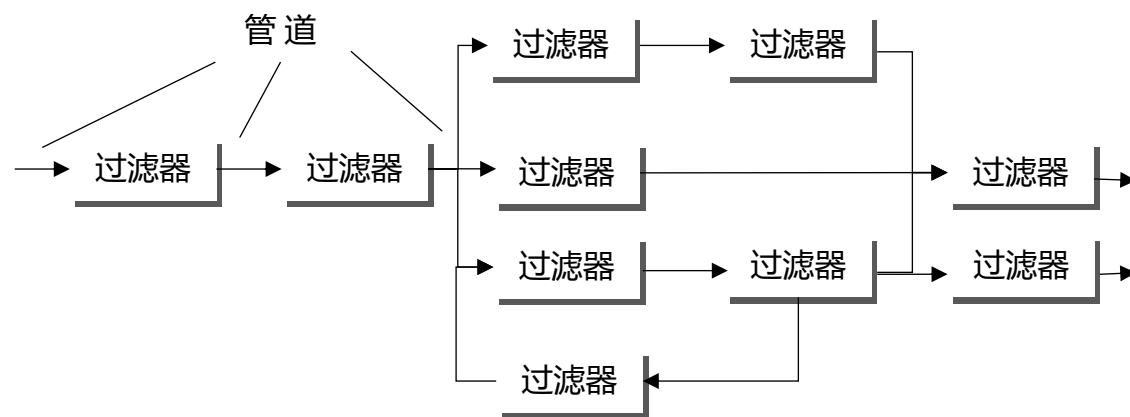
构件级设计细化描述每个模块，定义模块控制对象，形成程序代码



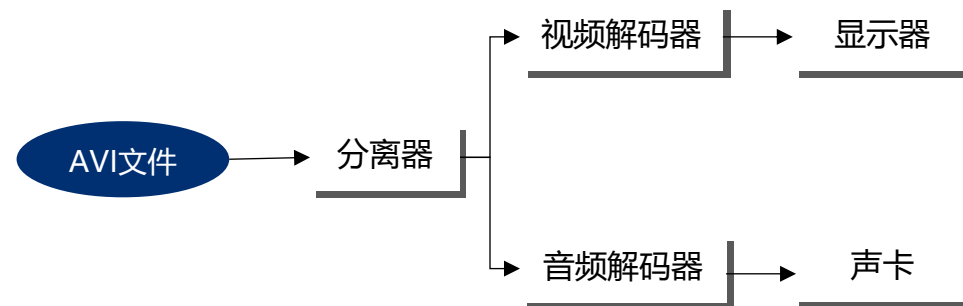
02.软件结构设计：体系结构设计

数据流体系结构--管道和过滤器模式

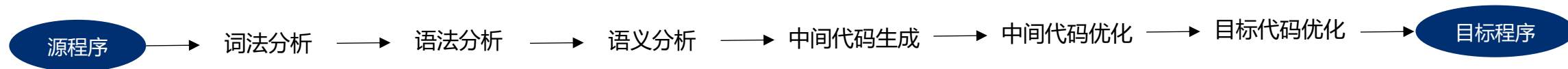
- 特点：输入数据经过一系列计算构件和操作构件的变换形成输出数据
- 应用：媒体播放器，编译器



数据流体系结构-管道过滤器模式



媒体播放器体系结构

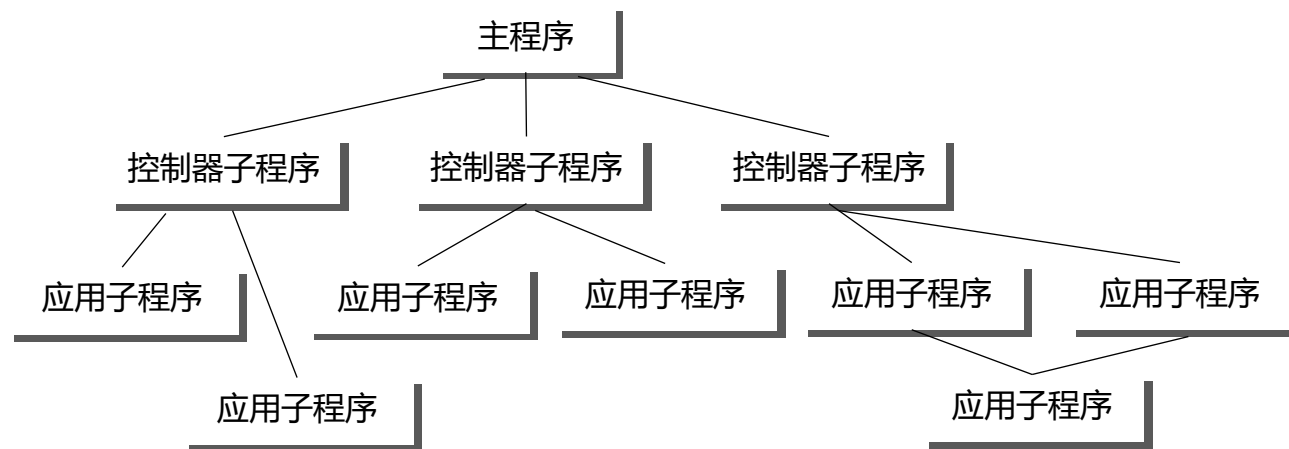


编译器体系结构

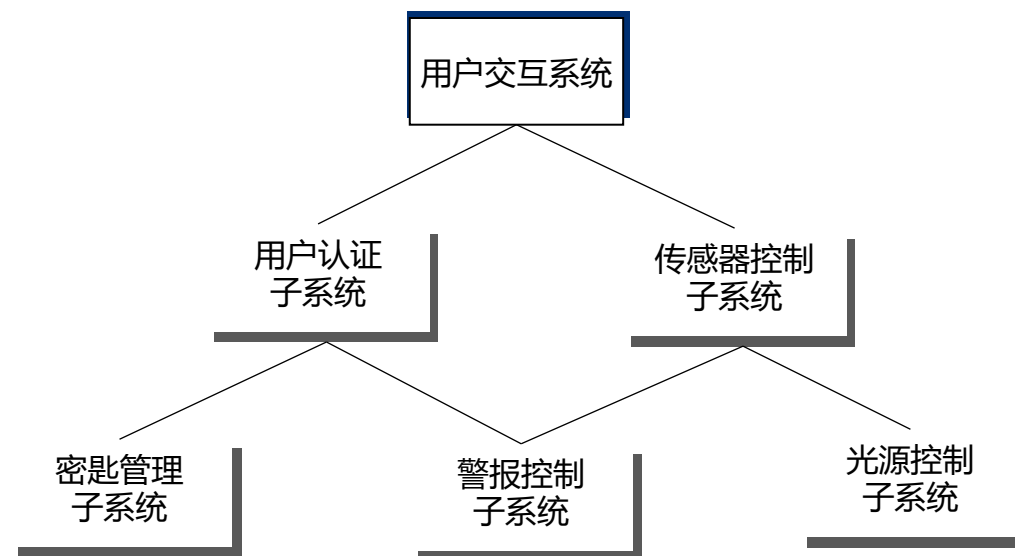
02.软件结构设计：体系结构设计

调用和返回体系结构--主程序/子程序模式

- 特点：目标功能被分解为一个控制层次，主程序调用一组程序构件，这些构件又去调用其他构件
- 应用：Safe Home Access软件



调用和返回体系结构 - 主程序/子程序模式



Safe Home Access软件系统体系结构

02.软件结构设计：构件级设计

构件是系统中模块化的、可部署的和可替换的部件，该部件封装了实现并对外提供一组接口

类是现实世界或思维世界中的实体在计算机中的反映，它将数据以及这些数据上的操作封装在一起定义为一种类型

对象是具有类类型的变量

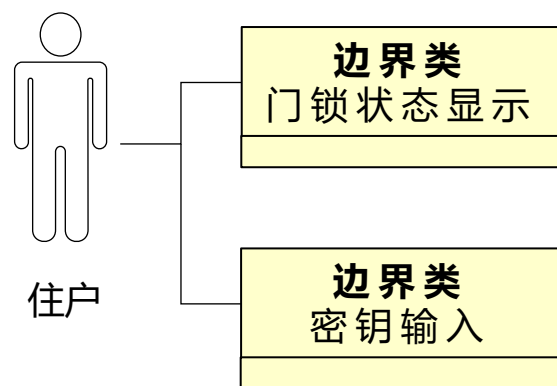
- 类是对象的抽象，而对象是类的具体实例。类是抽象的，不占用内存，而对象是具体的，占用存储空间。类是用于创建对象的蓝图，它是一个定义函数和变量的软件模板
- 在构件级设计中，每个类都会得到详细阐述，包括所有属性和与其实实现相关的操作

领域模型 (Domain Model)是对领域内的概念类或现实世界中对象的可视化表示

02.软件结构设计：构件级设计

Safe Home Access系统-住户解锁对应房间门

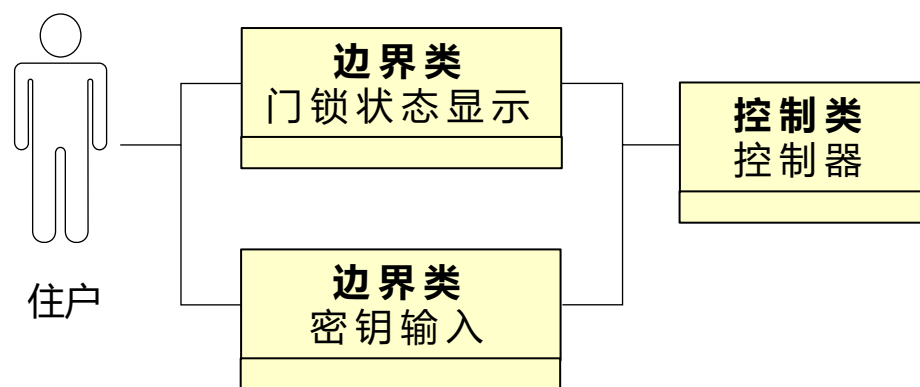
- 用户交互界面：密钥输入，门锁状态显示



02.软件结构设计：构件级设计

Safe Home Access系统-住户解锁对应房间门

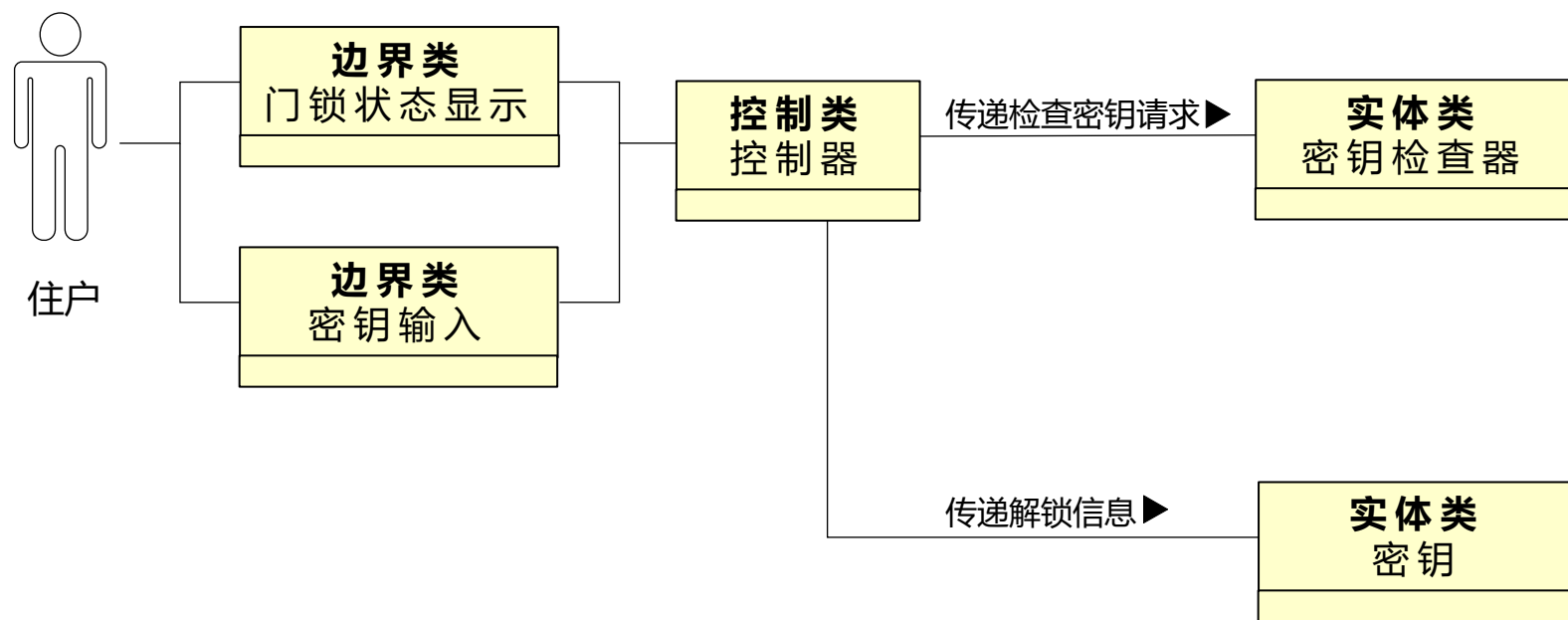
- 控制器协调与控制系统的内部构件，进行“发号施令”



02.软件结构设计：构件级设计

Safe Home Access系统-住户解锁对应房间门

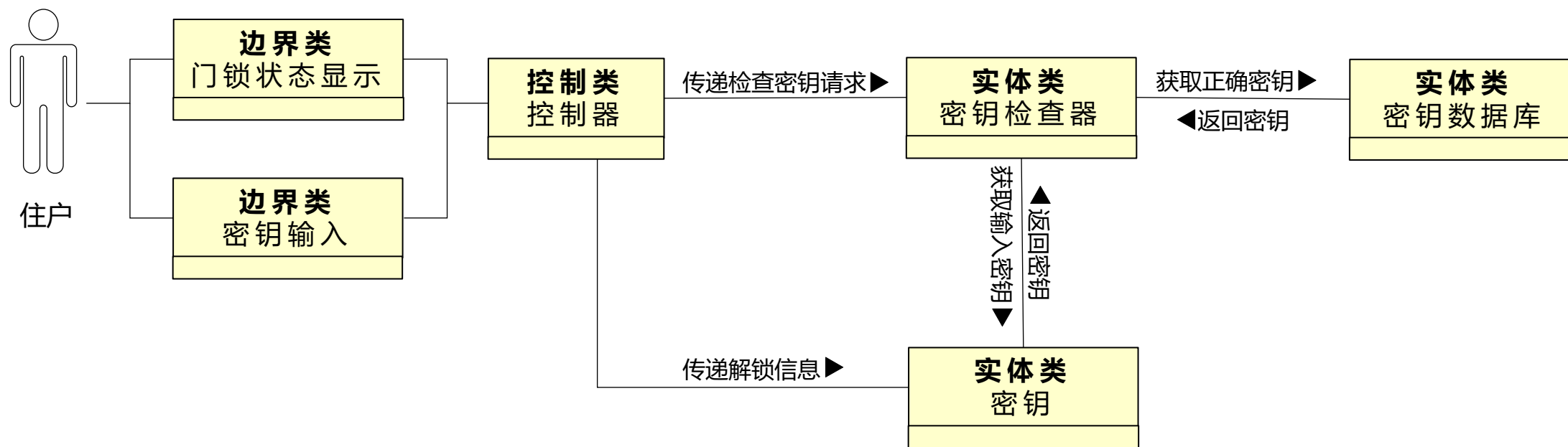
- 控制器向密钥类传递用户解锁信息，密钥类记录用户输入密钥和时间
- 控制器请求密钥检查器验证用户输入的键码是否有效



02.软件结构设计：构件级设计

Safe Home Access系统-住户解锁对应房间门

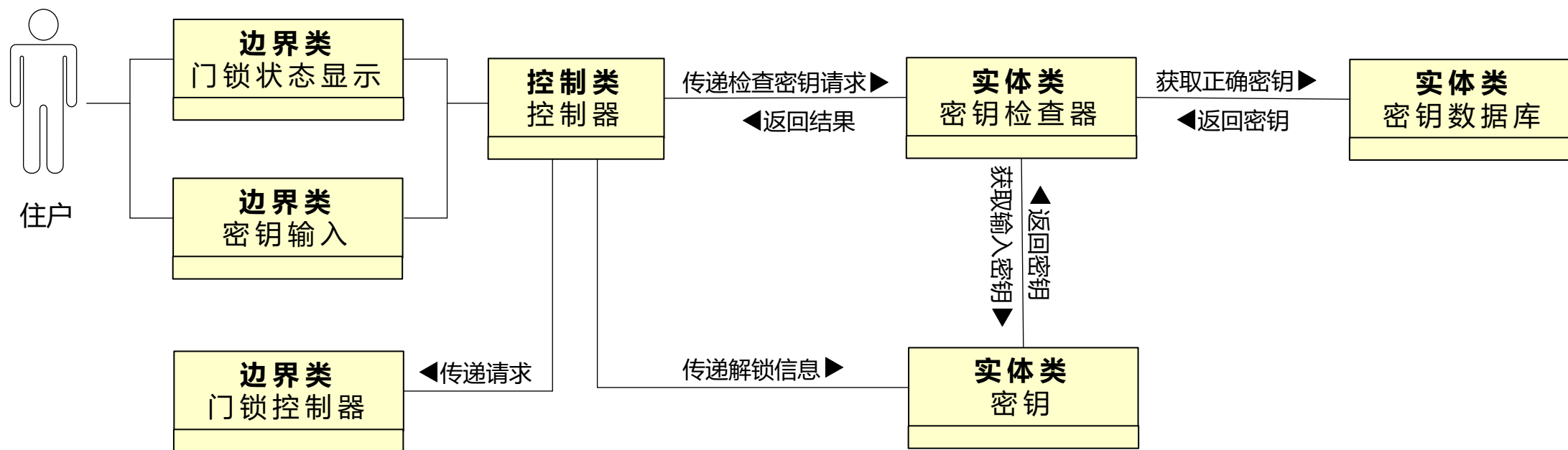
- 密钥检查器从密钥数据库获得正确密钥，从密钥获得用户输入密钥后验证用户输入密钥



02.软件结构设计：构件级设计

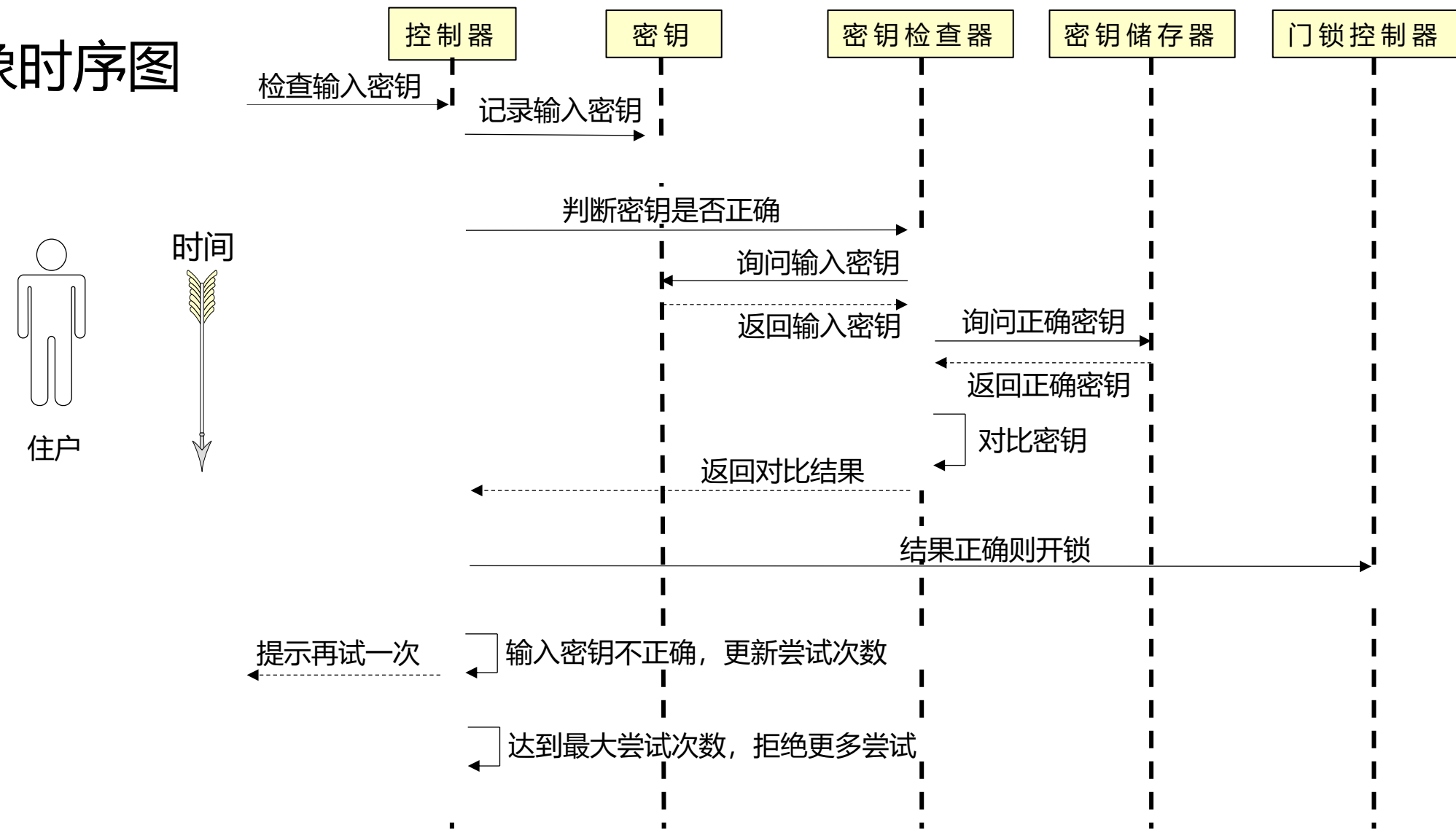
Safe Home Access系统-住户解锁对应房间门

- 验证结果返回控制器，正确控制器向门锁控制器传递开锁请求，否则检验输入次数提示重新输入



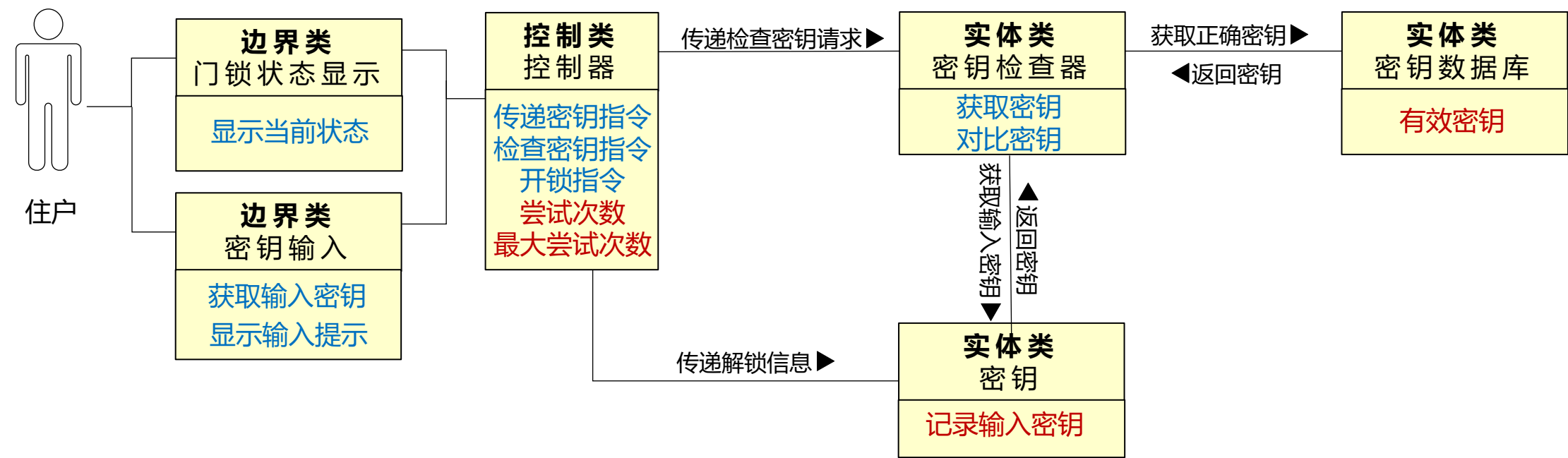
02.软件结构设计：构件级设计

对象时序图



02.软件结构设计：构件级设计

Safe Home Access系统-住户解锁对应房间门



目录

01

软件需求分析

02

软件结构设计

03

软件测试策略

03.软件测试策略

软件测试是为了发现程序中的错误而执行程序的过程

测试与软件开发各阶段的关系

- 软件开发过程是一个自顶向下，逐步细化的过程
- 测试过程是依相反的顺序安排的自底向上，逐步集成的过程

“测试不能表明软件中不存在错误，它只能说明软件中存在错误”

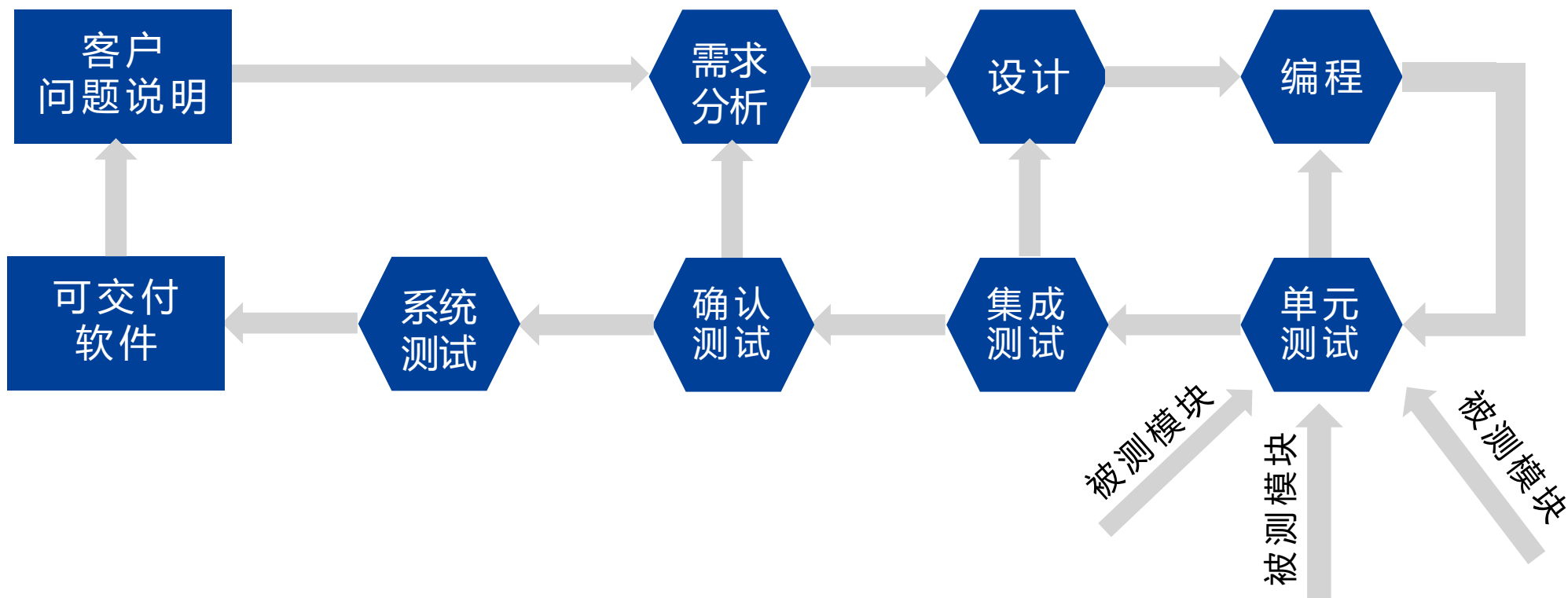
—艾兹赫尔·戴克斯特拉



艾兹赫尔·戴克斯特拉
Edsger W. Dijkstra
(1930-2002)

03.软件测试策略

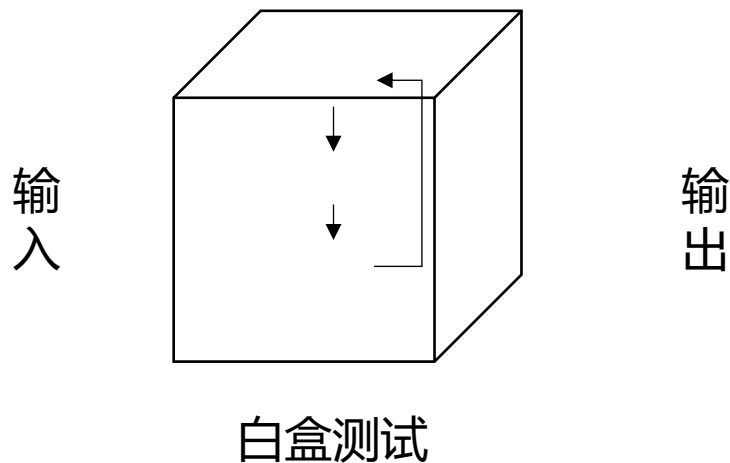
测试的过程按照四个步骤进行，即单元测试，集成测试，确认测试和系统测试



03.软件测试策略

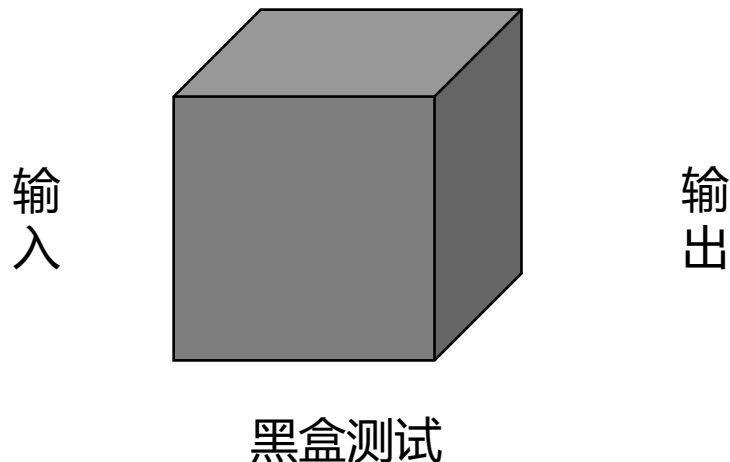
白盒测试（结构测试）

- 测试应用程序的内部结构或运作，而不是测试应用程序的功能
- 以编程语言的角度来设计测试案例。通过输入数据验证数据流在程序中的流动路径，并确定适当的输出



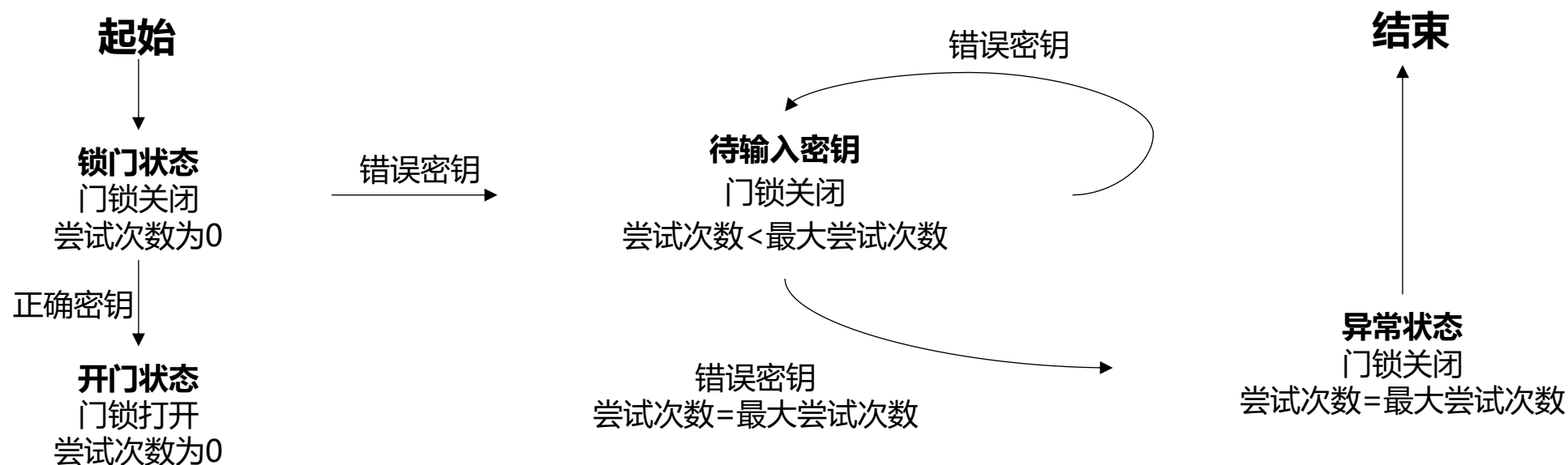
黑盒测试（功能测试）

- 通过测试来检测每个功能是否都能正常使用。在测试中，程序被看作不能打开的黑盒子，在完全不考虑程序内部结构和内部特性的情况下，在程序接口进行测试
- 黑盒测试只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当地接收输入数据而产生正确的输出信息



03.软件测试策略：白盒测试（结构测试）

基于状态图的测试定义了软件单元可以采用的一组抽象状态，并通过将其实际状态与预期状态进行比较来测试该单元的行为



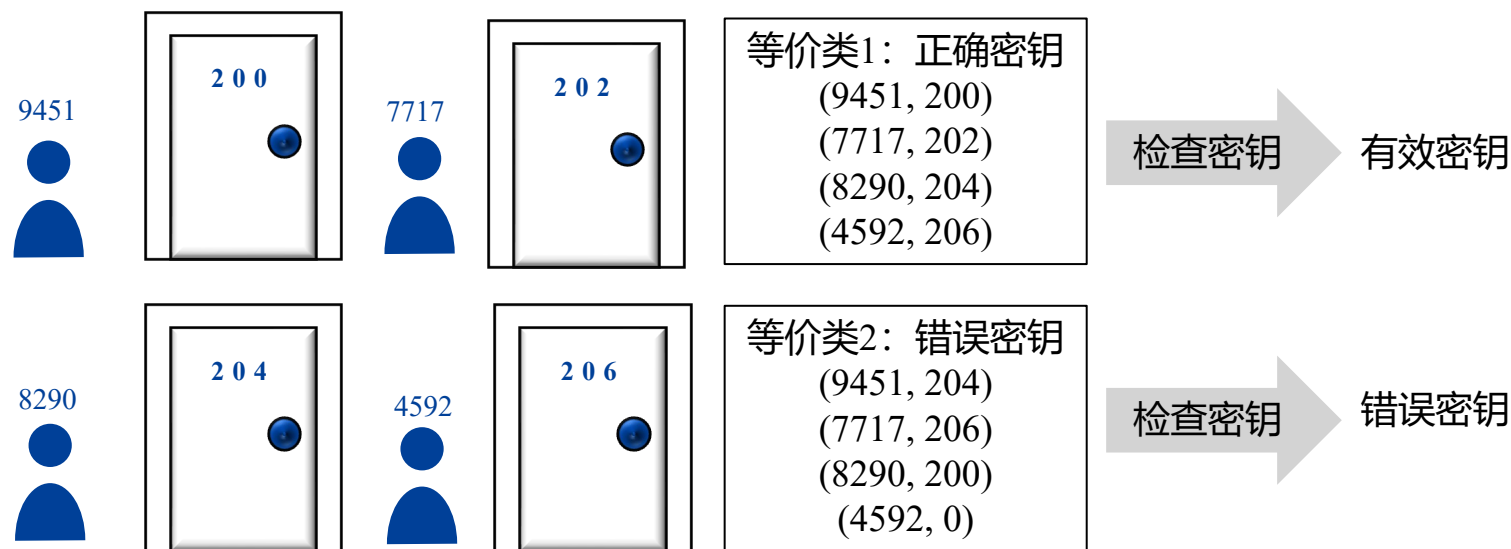
03.软件测试策略：黑盒测试（功能测试）

等价类划分方法

- 将所有可能的输入空间划分为等价类，以便期望程序在同一等价类的每个输入上有相同表现
- 步骤：(1) 将输入参数的值划分为等价类 (2) 从每个类中选择测试输入值

边界值划分方法：关注输入参数的边界值

- 从每个等价类的“边缘”或“异常值”中选择元素
例如 0，最小/最大值，空集，空字符串和null



文档-确保软件的正确使用和有效维护

用户文档

告诉用户如何一步步地使用软件包，通常包含教程指导用户熟悉软件包的各項特性

系统文档

- 定义软件本身，目的是为了let原始开发人员之外的人能够维护和修改软件包
- 系统文档需要记录软件开发的四个阶段
 - ◆ 分析：记录收集到的相关信息并记录信息的来源，需求和选用的方法必须用基于它们的推论来清楚表述
 - ◆ 设计：记录软件用到的工具，如最终版本结构图
 - ◆ 实现：记录代码的每个模块，代码应该使用注释和描述头尽可能详细地形成自文档化
 - ◆ 测试：记录每种测试的方法和结果

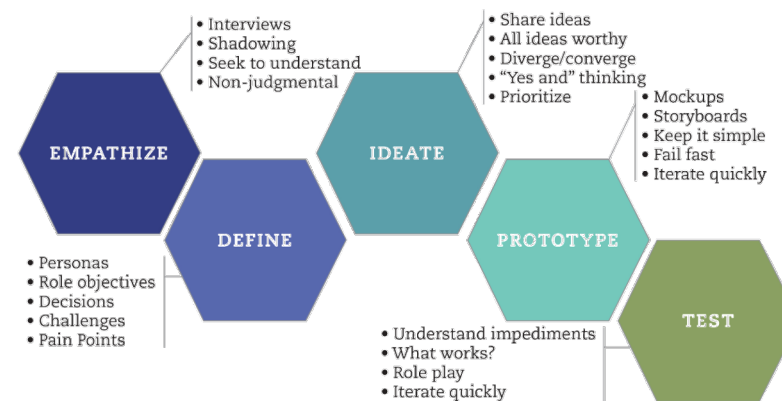
技术文档

技术文档描述软件系统的安装和服务，其中，安装文档描述软件如安装在每台计算机上，如服务器和客户端，服务文档描述了如果需要系统应该如何维护和更新

软件工程发展趋势（1）

用户体验的重要性不断提高

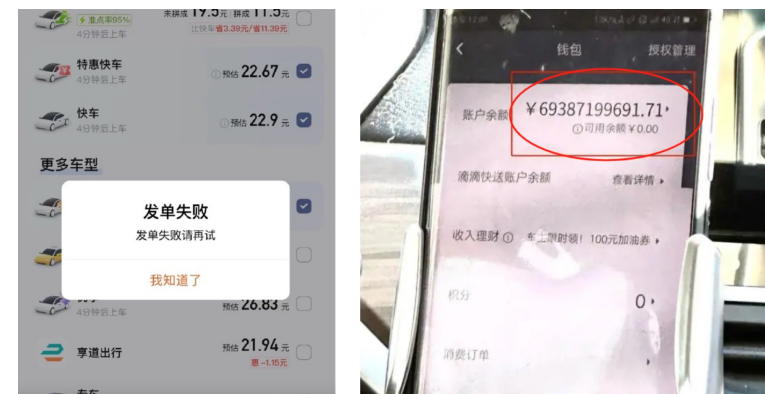
- 从以功能为中心转向以用户体验为中心
- 例如，设计思维（Design Thinking）模型要求以理解用户需求为出发点，定义问题，设计解决方案。



斯坦福大学设计思维模型

软件安全性和可靠性日益重要

- 事例：2023年11月，滴滴底层系统故障导致损失了超过1000万以上的订单和近4亿元的流水。



2023年11月27日滴滴打车系统故障

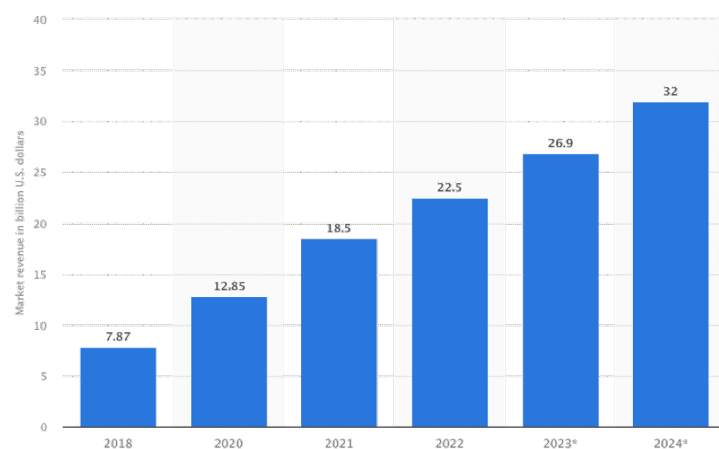
软件工程发展趋势（2）

低代码/无代码软件开发项目激增

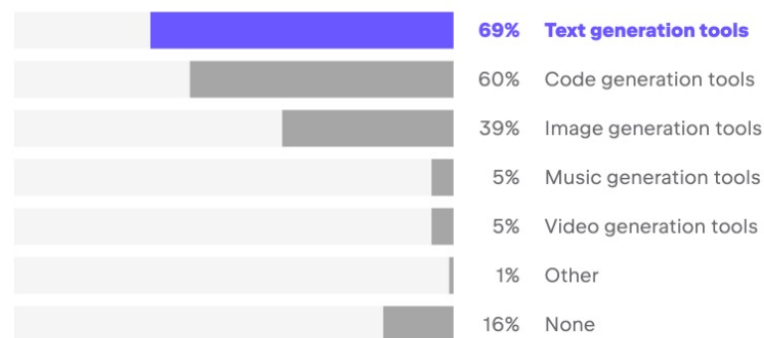
- 通过可视化/自然语言指令创建应用，摆脱对传统代码的依赖
- 降低开发门槛，促进灵活的业务操作

人工智能辅助的软件开发模式迅速崛起

- 以ChatGPT为代表的大语言模型，在软件开发任务上展现出卓越的性能
- 84%的开发者正在积极使用如大语言模型的生成式AI工具

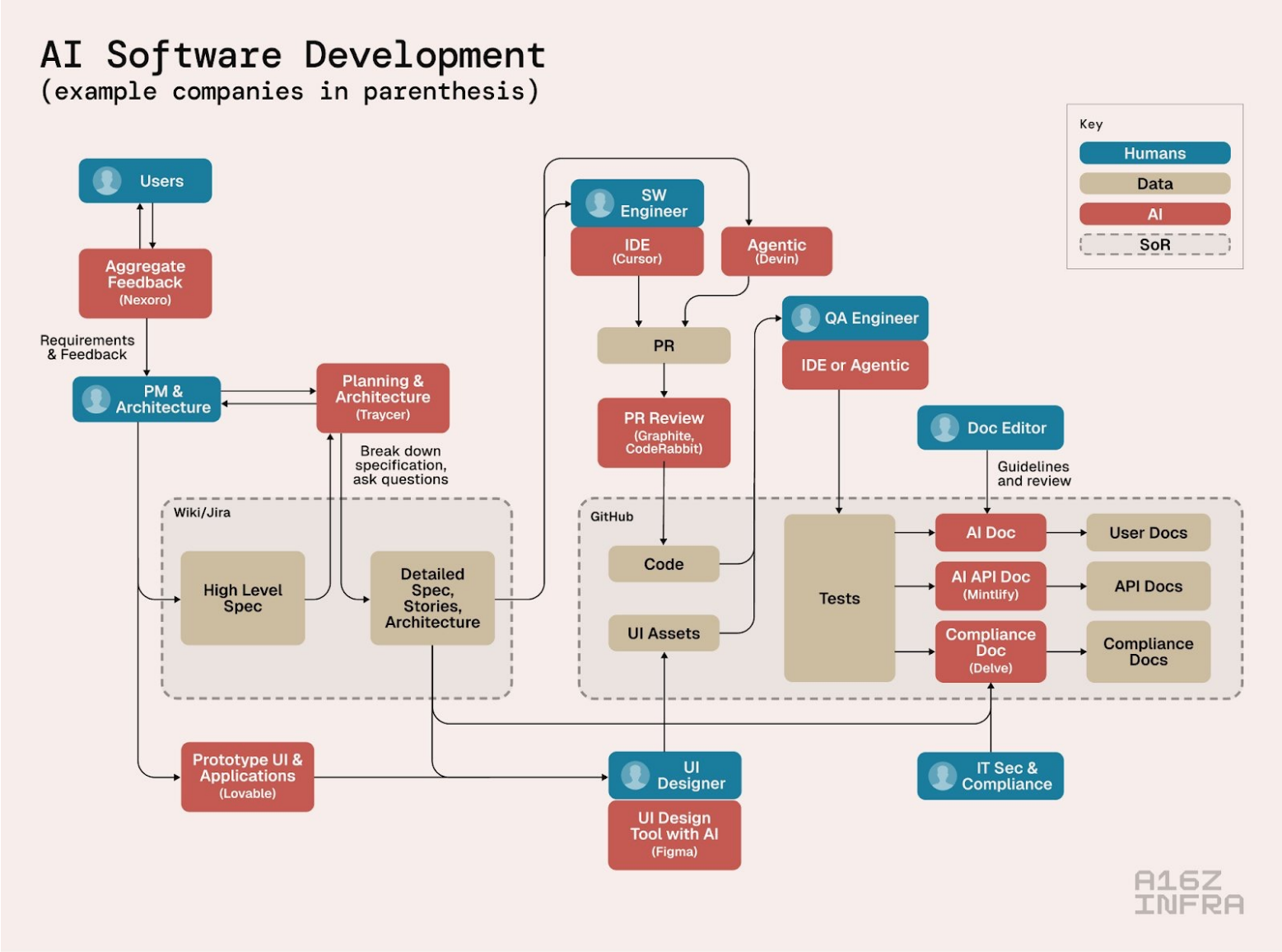


2018年至2024年全球低代码开发平台市场收入数据（数据来自Statista）



2023全球开发者生态调研（数据来自JetBrains）

大模型时代的软件工程: Vibe & Verification



人机协作

软件需求分析

- 需求分析过程
- 实例分析 Safe Home Access软件系统

软件工程中的设计

- 体系结构设计：以数据为中心的体系结构，数据流体系结构，调用和返回体系结构
- 构件级设计

软件测试策略

- 自顶向下，逐步细化
- 软件测试方法
 - ◆ 白盒测试：基于状态图的测试
 - ◆ 黑盒测试：等价类划分方法，边界值划分方法

谢谢!

